

# TORMENT MCP Cross-Host Research Findings

## Current TORMENT MCP surface and what matters for cross-host portability

TORMENT's MCP server is implemented as a **FastMCP (Python "mcp" SDK) stdio server** that exposes a **small, intentionally bounded tool surface** (one canonical "submit task" tool plus a handful of convenience wrappers) and a set of **read-only resources** for state/memory inspection. The project's own doctrine documents explicitly frame MCP as a **governed memory interface**, not an automation or tool-execution layer. <sup>1</sup> <sup>2</sup>

Two implementation choices are especially relevant when you plug TORMENT into multiple MCP hosts:

TORMENT returns a "governed envelope" as text. Every tool wrapper returns `json.dumps(...)` of a result object (including "blocked" decisions), rather than raising protocol-level errors. This is consistent with the project's "governance is an auditable outcome" posture, but it interacts with MCP's own error signalling conventions (see below). <sup>3</sup>

TORMENT is currently stdio-first. Cross-host success depends on the host's subprocess+stdio behaviour (encoding, buffering, stderr handling, timeouts, and how it renders tool results). MCP's stdio transport requirements are strict: newline-delimited JSON-RPC messages, UTF-8 encoding, and absolutely nothing except valid MCP messages written to stdout. <sup>4</sup>

One immediate operational footgun also surfaced: TORMENT imports the MCP Python SDK (`mcp.server.fastmcp`) but the `mcp` dependency is **not present in requirements.txt**, so "pip install -r requirements.txt" does not install the SDK needed for the MCP server to import at runtime. <sup>5</sup> <sup>6</sup>

## MCP protocol and Python SDK state as of April 2026

### Protocol version and negotiation

The **latest MCP spec version** is labelled **2025-11-25** on the official specification site, and MCP versions are negotiated during the `initialize` request via the `protocolVersion` field. The server may respond with a different supported version; if the client does not support the server's chosen version, it should disconnect. <sup>7</sup>

For HTTP transports, the spec also defines the `MCP-Protocol-Version` header on subsequent requests and a fail-fast behaviour for invalid/unsupported versions (HTTP 400). The spec additionally documents a backwards-compatibility default of assuming **2025-03-26** if a server cannot otherwise identify the version. <sup>4</sup>

A major transport-level break between 2024–2026 is the move from **HTTP+SSE** in older versions (notably 2024-11-05) to **Streamable HTTP** as the replacement transport in later versions. <sup>4</sup>

## Tool results, structuredContent, and error signalling

In the 2025-11-25 spec, a `tools/call` response returns a **Tool Result** object containing a `content` array plus an optional `isError` boolean. The spec explicitly distinguishes:

Protocol errors: JSON-RPC error responses for malformed requests, unknown tools, and server/protocol failures.

Tool execution errors: reported as normal tool results with `isError: true`, intended to be visible to the model so it can self-correct and retry. <sup>8</sup>

The spec also introduces **structuredContent**: a structured JSON object in `structuredContent`, and it recommends that tools returning structured content also include a serialised JSON blob in a text content block for backwards compatibility. <sup>9</sup>

The official Python SDK documentation aligns with the spec:

Tools should signal expected errors with `ToolError`, which is returned to the client as a tool result with `isError=True`; unhandled exceptions are also converted into `isError=True` tool results; and advanced implementations can return a `CallToolResult` directly to control `content`, `structuredContent`, and `_meta`. <sup>10</sup>

Structured output is now a first-class feature: if a tool's return type annotation is compatible, the SDK produces structured results (and still provides unstructured results for backwards compatibility and "quirks-compatibility"). <sup>11</sup>

These details matter for TORMENT because its "governed envelope" approach is currently implemented as a **JSON string** returned from each tool wrapper. That remains valid as unstructured text content in MCP, but it does not exploit `structuredContent`, and it does not map "blocked" governance outcomes onto `isError` (unless TORMENT explicitly chooses to do so). <sup>3</sup> <sup>8</sup>

## FastMCP status and whether it is deprecated

FastMCP (`from mcp.server.fastmcp import FastMCP`) is still the canonical "quickstart" path in the official SDK in April 2026:

The PyPI package `mcp` (Anthropic-authored) is at **1.27.0**, released **April 2, 2026**, and the quickstart example instantiates `FastMCP(..., json_response=True)` and runs `transport="streamable-http"`. <sup>6</sup>

The official Python SDK docs continue to present FastMCP as the primary "server" abstraction and document server construction, tools/resources/prompts, structured output, and error handling using FastMCP. <sup>12</sup>

Verdict: **FastMCP is not deprecated** in the official SDK as of April 2026; it is actively documented and used as the primary ergonomic server API. <sup>13</sup>

One ecosystem caveat: there is also a separate “fastmcp” ecosystem in the wild (different branding/sites). TORMENT, however, is using the official `mcp` SDK import path (`mcp.server.fastmcp`). The key portability risk is not “FastMCP deprecation” but “SDK not installed/pinned” (see risks). <sup>3</sup> <sup>5</sup> <sup>6</sup>

## Hermes Agent identification and a practical connection cookbook for TORMENT

### Which “Hermes” is relevant for MCP hosts

In the MCP-host context you described, “Hermes” most plausibly refers to **Hermes Agent (Nous Research)**, which has first-class MCP client support: it can connect to local stdio MCP servers (`command` / `args` / `env`) and remote MCP servers (`url` / `headers`), discovers tools at startup, and offers per-server filtering. <sup>14</sup>

This is distinct from unrelated software also named “Hermes” (for example the IBC relay CLI), so documentation and configuration sources must be taken from the Hermes Agent documentation site. <sup>15</sup>

Canonical project link (not analysed beyond identifying the project):

```
https://github.com/NousResearch/hermes-agent
```

(That repository URL is linked from the Hermes Agent docs UI.) <sup>16</sup>

### Critical Windows-side reality for Hermes

Hermes Agent explicitly states that **native Windows is not supported**; Windows users are instructed to run it under **WSL2**. This matters because stdio MCP servers are launched as subprocesses, so your “Hermes ↔ TORMENT” stdio connection is simplest when both run inside the same environment. <sup>17</sup>

### Hermes MCP configuration shape

Hermes reads MCP configuration from `~/.hermes/config.yaml` with an `mcp_servers:` mapping. Each server entry supports either stdio (`command`, `args`, `env`) or HTTP (`url`, `headers`), plus timeouts and tool filtering. Hermes also sanitises names (hyphens/dots become underscores) and prefixes registered tool names as `mcp_<server>_<tool>`. <sup>14</sup>

Hermes security posture includes **stdio environment filtering**: it does not pass your entire shell environment by default; it passes explicitly configured `env` plus a baseline. This is good for safety, but it means TORMENT must receive all required env vars via the MCP config entry (or it may start and then fail at runtime). <sup>18</sup>

### Hermes ↔ TORMENT cookbook (WSL2-friendly)

This cookbook assumes you want Hermes Agent (as MCP host) to consume TORMENT (as MCP server) over stdio, matching TORMENT’s current MCP transport.

Install/run Hermes in WSL2. Hermes' official install instructions recommend Linux/macOS/WSL2 only.

17

Ensure TORMENT is runnable as a Python stdio server inside the same environment. TORMENT's MCP server code uses the FastMCP server and runs as a stdio MCP server process. 3 4

Install the MCP Python SDK in the TORMENT environment. The official SDK is `pip install mcp` (or `pip install "mcp[cli]"`), and it requires Python  $\geq 3.10$ . 6  
(You will likely want to add `mcp` to TORMENT requirements so users can install it reliably. 5 )

Add a TORMENT MCP server entry to Hermes' `~/.hermes/config.yaml`. Example shape (you will adjust paths/venv/module name to your repo layout):

```
mcp_servers:
  torment:
    command: "python3"
    args:
      - "-m"
      - "torment_service.mcp_server"
    env:
      # Whatever TORMENT needs to boot correctly in server mode.
      # Hermes will not pass your whole shell env through by default.
      TORMENT_WORKSPACE_ID: "default"
      TORMENT_AGENT_ID: "default"
      TORMENT_MCP_EXPOSURE_TIER: "open"
      # Add other required env vars explicitly.
    timeout: 120
    connect_timeout: 60
    tools:
      # Optional: allowlist just the core memory interface first
      include: ["torment_query_memory", "torment_ingest",
"torment_feedback", "torment_submit_task"]
      resources: true
      prompts: false
```

The key is that Hermes' config shape and filtering semantics match this `mcp_servers` schema. 19

Restart Hermes and confirm tool discovery. Hermes discovers MCP servers at startup and registers their tools with a prefix (e.g., `mcp_torment_torment_query_memory`). 14

Validate the stdio transport invariants. MCP stdio messages must be newline-delimited JSON-RPC, UTF-8 encoded, and the server must never write non-MCP content to stdout; stderr may be used for logs. 4

Because Hermes runs in WSL2 and TORMENT is currently stdio-only, "Hermes in WSL2  $\leftrightarrow$  TORMENT on native Windows" is possible but brittle (cross-environment subprocess + stdio). You will get a far more reliable dev experience by co-locating both processes in WSL2 or moving TORMENT to an HTTP transport later. The protocol supports Streamable HTTP and many hosts support it, but that would be a separate TORMENT implementation decision. 20

# Portability risks, a Windows stdio checklist, and cross-host config patterns

## Portability risks in Risk / Symptom / Mitigation format

SDK dependency missing

Risk: TORMENT MCP server fails to import/run because `mcp` is not installed from requirements.

Symptom: `ModuleNotFoundError: No module named 'mcp'` when starting the server.

Mitigation: Add `mcp` (or `mcp[cli]` if needed) to dependency management and pin to a compatible range; verify Python  $\geq$  3.10. <sup>5</sup> <sup>6</sup>

Protocol version mismatch between host and server

Risk: A host using only newer protocol versions refuses a server that negotiates older ones (or vice-versa).

Symptom: Disconnect during `initialize` with an “unsupported protocol version” path.

Mitigation: Ensure TORMENT’s MCP SDK is recent; MCP’s lifecycle and version negotiation are explicit and designed to fail fast if no overlap exists. <sup>21</sup>

Stdout contamination

Risk: Any non-MCP output written to stdout breaks the session.

Symptom: Host reports protocol parse errors; tools/resources never appear.

Mitigation: Keep logs on stderr only; audit dependencies for accidental prints; MCP stdio forbids non-MCP stdout. <sup>4</sup>

Governed “blocked” outcomes are not surfaced as tool errors in host UX

Risk: Hosts render TORMENT’s `{"ok": false, ...}` envelope as a normal tool success, reducing clarity (or causing automated flows to treat governance rejection as success).

Symptom: A blocked request appears as plain JSON text rather than an obvious error state.

Mitigation: Consider mapping “blocked” outcomes to MCP tool execution errors (`isError: true`) while still returning the full governed envelope in content/structuredContent; MCP explicitly supports tool execution errors distinct from protocol errors. <sup>22</sup>

Structured output not utilised

Risk: Some hosts increasingly expect or prefer structured display/handling, especially when they validate output schemas.

Symptom: Host shows raw JSON string; no structured UI or post-processing.

Mitigation: Return typed dict/Pydantic models and allow the SDK to emit `structuredContent` with back-compat unstructured text; the SDK documents this as the default structured output behaviour. <sup>23</sup>

Environment variable filtering (host-side) hides required configuration

Risk: Hermes (and some other hosts) do not pass the full env by default; missing env causes TORMENT to boot incorrectly or behave unexpectedly.

Symptom: TORMENT starts but fails to access expected workspace/agent config or fails later.

Mitigation: Document required env vars per host and include them explicitly in config; Hermes supports explicit per-server env and filtering semantics. <sup>19</sup>

Tool output size limits and token warnings

Risk: Large responses (e.g., returning long memory dumps) hit host UX limits or warnings.

Symptom: Claude Code warns when tool output exceeds a token threshold (documented as 10,000 tokens).

Mitigation: Prefer summaries by default, add pagination/filters, and keep resource outputs bounded.

24

Windows console encoding and buffering can corrupt stdio if you run servers manually in a console

Risk: Non-UTF-8 console code pages and buffered I/O can cause mojibake or deadlocks when redirected or wrapped.

Symptom: Garbled characters; host timeouts; stalled handshake.

Mitigation: Prefer host-spawned subprocess pipes; use unbuffered mode where appropriate; understand Windows code pages and Python stdio encoding rules (see checklist). 25

SIGTERM/SIGKILL shutdown semantics differ on Windows

Risk: MCP lifecycle guidance for stdio mentions SIGTERM/SIGKILL as escalation; Windows hosts handle termination differently.

Symptom: Servers linger, or host kills abruptly without graceful shutdown.

Mitigation: Ensure server tolerates abrupt stdin closure; treat shutdown as “transport closes” outcome. MCP lifecycle explicitly frames stdio shutdown beginning with closing the input stream. 21

## Windows stdio checklist for Python MCP servers (actionable and source-anchored)

The MCP spec requires stdio JSON-RPC messages to be UTF-8 and newline-delimited, with no embedded newlines in the message framing (newlines inside JSON strings must be escaped). stdout must contain only MCP messages; stderr may be used for logs. 4

If you are launching Python MCP servers on Windows:

Use unbuffered stdout/stderr for safety. Python’s `-u` forces stdout/stderr unbuffered and is explicitly documented for stream flushing behaviour. 26

Control encoding intentionally when you are attached to a Windows console. Python documents that Windows console encoding behaviour is special; overriding encoding via `PYTHONIOENCODING` on Windows console streams has caveats unless legacy console stdio mode is used. 27

Know (or fix) the active console code page if you run interactively. Windows’ `chcp` command changes the active console code page and affects how programs started after the change interpret output. 28

Prefer host-spawned pipe stdio over “run in a console then attach”. In stdio MCP, the host launches the server subprocess and manages stdin/stdout directly. That bypasses most console encoding pitfalls and is the normative transport model. 29

## Cross-host configuration patterns you can reuse

Claude Code. Claude Code offers an MCP management CLI (`claude mcp ...`), supports multiple scopes (`local`, `project`, `user`), and uses `.mcp.json` for project-scoped sharing; it also supports importing server configs from Claude Desktop (macOS/WSL). 24

Continue. Continue stores MCP server blocks under `.continue/mcpServers`, supports YAML blocks, and can also directly consume JSON MCP config formats from tools like Claude Desktop, Cursor, and Cline. 30

Cline. Cline stores MCP settings in `cline_mcp_settings.json` and supports both stdio (`command`, `args`, `env`) and remote (`url`, `headers`) server entries; it also documents host-side allowlists (`alwaysAllow`) and a `disabled` switch. <sup>31</sup>

Hermes Agent. Hermes stores MCP config under `~/.hermes/config.yaml`, supports stdio and HTTP, per-server include/exclude filters, optionally disables resources/prompts wrappers, exposes OAuth options for HTTP servers, and sanitises/ prefixes tool names. <sup>19</sup>

## Direct answers to the research questions (Q1–Q21) with confidence labels

Q1 — What is “Hermes” in MCP-host sense?

Answer: Hermes Agent (Nous Research) is a prominent MCP-capable agent host; its docs explicitly describe connecting to MCP servers under `mcp_servers` and installing MCP support via an extra. Confidence: High. <sup>32</sup>

Q2 — How does Hermes configure stdio MCP servers?

Answer: Hermes uses `~/.hermes/config.yaml` with an `mcp_servers:` mapping; stdio servers are defined with `command`, `args`, and `env`, plus timeouts and filtering. Confidence: High. <sup>14</sup>

Q3 — How does Hermes configure HTTP MCP servers and auth?

Answer: Hermes supports HTTP servers via `url` and `headers`; it also documents an `auth` key (e.g., OAuth 2.1 with PKCE) for HTTP servers and persists tokens under `~/.hermes/mcp-tokens/`. Confidence: High. <sup>33</sup>

Q4 — Does Hermes filter env vars for stdio servers?

Answer: Yes; Hermes documents that stdio env is filtered and only explicit `env` plus a safe baseline are passed to subprocesses. Confidence: High. <sup>18</sup>

Q5 — MCP portability gotchas for stdio across hosts (protocol-level)

Answer: Message framing is newline-delimited JSON-RPC, UTF-8 encoded; no embedded newlines in the message framing; stdout must be MCP messages only; stderr logging is allowed; client launches server as subprocess. Confidence: High. <sup>29</sup>

Q6 — Is TORMENT’s current MCP server shape “Claude-Desktop-specific”?

Answer: The underlying server is standard MCP stdio (so it should be host-agnostic), but TORMENT’s docs and smoke test are framed around Claude Desktop workflow and configuration (project docs emphasise Claude Desktop integration as the primary example). Confidence: Medium-High. <sup>34</sup> <sup>35</sup>

Q7 — What other MCP hosts are widely used (evidence-backed list)

Answer (evidence in this research): Claude Code is an MCP host with explicit documentation and management commands. Continue supports MCP server blocks and explicitly references Claude Desktop/Cursor/Cline config formats. Cline supports MCP and documents its config schema. Hermes supports MCP as host. Confidence: High for these four. <sup>36</sup>

Note: Cursor almost certainly supports MCP in practice, but its official docs could not be reliably crawled in this session; Continue’s docs nonetheless reference Cursor as a source of JSON MCP configs.

Confidence: Medium. <sup>37</sup>

Q8 — Is there a canonical host-compatibility matrix?

Answer: The official MCP Registry is focused on server metadata and explicitly is “not intended to be directly consumed by host applications” (hosts are expected to consume registries/aggregators), and registry docs note the registry is currently for servers (with possible future extension to clients/hosts). A canonical, official cross-host compatibility matrix is not presented there. Confidence: High. <sup>38</sup>

Q9 — Is there a protocol “version string” and how is it negotiated?

Answer: Yes; protocol versions are date-stamped (e.g., 2025-11-25) and negotiated via the `initialize` request `protocolVersion` field; the server replies with the chosen version and the client disconnects if unsupported. Confidence: High. <sup>21</sup>

Q10 — How should mismatched protocol versions fail?

Answer: MCP specifies an explicit initialise-time mismatch handling (client disconnects if it cannot support server’s chosen version) and (for HTTP) mandates 400 for invalid/unsupported `MCP-Protocol-Version` headers. Confidence: High. <sup>21</sup>

Q11 — How host-agnostic is the smoke test conceptually?

Answer: A good smoke test should validate tools/resources listing, a basic ingest/query loop, and error/result rendering without assuming a particular host UI. This is consistent with MCP’s protocol surface (tools/list, tools/call, resources/list/read) and lifecycle requirements. Confidence: Medium (principled inference from spec). <sup>39</sup>

TORMENT’s existing smoke test is valuable but framed around Claude Desktop-style configuration and expectations. Confidence: Medium-High. <sup>34</sup>

Q12 — What terminology should the positioning use?

Answer: “Memory layer” is already a highly populated phrase (e.g., Mem0 brands itself as “the Memory Layer for AI Apps”), so TORMENT should differentiate with governance/provenance framing (suggestions below). Confidence: High. <sup>40</sup>

Q13 — FastMCP status (recommended, deprecated, Windows issues)

Answer: FastMCP is still the official primary server abstraction in the MCP Python SDK; it is actively documented and used in the PyPI quickstart (mcp 1.27.0, Apr 2, 2026). Windows-specific issues are largely about subprocess stdio encoding/buffering rather than FastMCP being deprecated. Confidence: High. <sup>41</sup>

Q14 — Protocol revision state and breaking changes since 2024–2025

Answer: MCP 2025-11-25 is tagged “latest”; Streamable HTTP replaces the older HTTP+SSE transport from 2024-11-05; version negotiation is part of the initialise lifecycle. Confidence: High. <sup>29</sup>

Q15 — Structured content compliance vs “JSON string in text content”

Answer: MCP supports both unstructured `content` (TextContent etc.) and optional `structuredContent`. Returning JSON as text remains valid as unstructured output; for structured output, the spec recommends also returning a serialised JSON text block for backwards compatibility. The Python SDK documents structured output as default when return types are compatible and still returns unstructured output for backwards compatibility/quirks. Confidence: High. <sup>42</sup>

Q16 — Schema strictness (tool names and JSON Schema dialect expectations)

Answer: The spec recommends tool name length 1–128 and a constrained character set; it also requires valid JSON Schema objects for inputSchema and defaults schema dialect to 2020-12 when `$schema` is absent while requiring clients/servers to support 2020-12. Confidence: High. <sup>43</sup>

Host-specific additional strictness varies and should be validated per host; this research prioritised primary sources and did not exhaustively benchmark every host's validator in practice. Confidence: Medium.

#### Q17 — Envelope-with-ok-false vs MCP tool errors

Answer: MCP explicitly provides `isError: true` as a tool execution error channel distinct from protocol errors, intended to be visible to the model. The Python SDK provides `ToolError` and direct `CallToolResult` returns to set this flag while still returning usable content/structuredContent. Therefore, TORMENT's governed envelope pattern is compatible, but if you want host UX to clearly render "blocked" responses as errors, mapping some governance denials to `isError` is MCP-native. Confidence: High. <sup>22</sup>

#### Q18 — Windows stdio gotchas checklist (encoding/buffering/code page)

Answer: MCP requires UTF-8 JSON-RPC messages; Windows console code pages influence text encoding; `chcp` changes the active console code page; Python documents stdout buffering and `-u` / `PYTHONUNBUFFERED` and caveats for `PYTHONIOENCODING` on Windows console streams. Confidence: High. <sup>44</sup>

#### Q19 — "Memory layer for agents" landscape (selected projects)

Answer: Mem0 positions itself as a general "memory layer" and also discusses MCP integration; Zep provides a Memory API and high-level agent memory; Letta (MemGPT lineage) documents a two-tier memory hierarchy; Cognee explicitly framed MCP as a bridge to its memory layer; Honcho positions itself as persistent cross-session memory for Hermes; LangChain and LlamaIndex both document memory subsystems in their agent frameworks. Confidence: High (sources below). <sup>45</sup>

#### Q20 — "Spine" terminology collision

Answer: Within the sources surveyed here, "Spine" is not a dominant brand term for the main MCP host projects or the listed memory-layer projects; however, this was not exhaustively searched across the full ecosystem. Confidence: Medium-Low (flagged uncertainty). (No load-bearing primary source located in this run.)

#### Q21 — Lower-priority protocol surface (auth, sampling, resources, size limits)

Answer: Hermes supports OAuth 2.1 with PKCE for HTTP servers; MCP spec includes authorisation framework for HTTP and capability negotiation during initialise; Hermes documents sampling support (`sampling/createMessage`) and runtime tool list change notifications; Claude Code documents warnings for large tool outputs and support for `list_changed` notifications. Confidence: High. <sup>46</sup>

## Positioning language and a compact landscape table

### Positioning phrases that avoid tagline collision while matching what developers search for

The ecosystem already uses "memory layer" heavily (Mem0 explicitly brands itself this way). <sup>40</sup>

To remain discoverable without becoming indistinguishable, these phrases keep search relevance while foregrounding the differentiator (governance + provenance + auditability):

Governed memory layer for MCP agents

Rationale: keeps the "memory layer" keyword but adds the governance boundary that most competitors do not emphasise as first-class.

Provenance-first memory core

Rationale: “provenance” is a strong differentiator and aligns with TORMENT’s doctrine posture; it also hints at auditability and safe ancestry.

Audit-trailed memory engine

Rationale: communicates operational value to teams integrating agents into production systems (debugging, compliance, post-mortems).

Policy-aware memory interface

Rationale: frames “blocked” outcomes as a feature rather than a failure, and matches MCP’s bias toward human-in-the-loop tool governance. <sup>47</sup>

MCP memory layer with bounded capability surface

Rationale: directly echoes MCP’s host/server separation and fits TORMENT’s doctrine of “remember tool outputs, don’t execute tools” as the core boundary. <sup>1</sup>

## **Landscape table (max 8) focused on “memory layer” terminology and integration paths**

Mem0 — explicitly brands itself as a memory layer; markets reduced token costs, continuity, and MCP integration paths. <sup>48</sup>

Zep — provides a Memory API and high-level agent memory with session/user modelling and retrieval patterns. <sup>49</sup>

Letta (MemGPT lineage) — focuses on memory hierarchy (core/in-context vs archival/out-of-context), self-editing and structured agent memory management. <sup>50</sup>

Cognee — explicitly positions MCP as a bridge between models/agents and its AI memory layer. <sup>51</sup>

Honcho — positioned as a cross-session memory and user modelling layer; integrates with Hermes in a “two memory layers” model. <sup>52</sup>

LangChain / LangGraph memory — documents short-term vs long-term memory, stores, and scoped memory concepts inside an agent framework. <sup>53</sup>

LlamaIndex memory — documents a Memory class supporting short- and long-term memory and deprecates older buffers in favour of more flexible memory abstractions. <sup>54</sup>

Hermes built-in memory (plus MCP client support) — Hermes stores local memory in `~/ .hermes/` and provides MCP hosting; it also supports an external memory layer (Honcho) alongside built-in memory, which is structurally similar to what TORMENT is trying to be for agents. <sup>55</sup>

---

<sup>1</sup> MCP\_CAPABILITY\_BOUNDARY.md

[https://github.com/pzychozen/TORMENT/blob/main/docs/MCP\\_CAPABILITY\\_BOUNDARY.md](https://github.com/pzychozen/TORMENT/blob/main/docs/MCP_CAPABILITY_BOUNDARY.md)

<sup>2</sup> PATH\_3\_MCP\_DEVELOPER\_EXPERIENCE.md

[https://github.com/pzychozen/TORMENT/blob/main/docs/PATH\\_3\\_MCP\\_DEVELOPER\\_EXPERIENCE.md](https://github.com/pzychozen/TORMENT/blob/main/docs/PATH_3_MCP_DEVELOPER_EXPERIENCE.md)

3 torment\_service/mcp\_server.py

[https://github.com/pzychozen/TORMENT/blob/23d6fb43068d6471e9486460a14741c04c16c849/torment\\_service/mcp\\_server.py](https://github.com/pzychozen/TORMENT/blob/23d6fb43068d6471e9486460a14741c04c16c849/torment_service/mcp_server.py)

4 20 25 29 44 Transports - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>

5 requirements.txt

<https://github.com/pzychozen/TORMENT/blob/main/requirements.txt>

6 13 41 mcp · PyPI

<https://pypi.org/pypi/mcp>

7 21 39 Lifecycle - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle>

8 9 22 42 43 47 Tools - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-11-25/server/tools>

10 11 23 Building Servers - MCP Server

<https://py.sdk.modelcontextprotocol.io/server/>

12 MCP Server

<https://modelcontextprotocol.github.io/python-sdk/>

14 16 18 32 MCP (Model Context Protocol) | Hermes Agent

<https://hermes-agent.nousresearch.com/docs/user-guide/features/mcp/>

15 Configure Hermes - Hermes (IBC Relay CLI) Documentation

[https://hermes.informal.systems/documentation/configuration/configure-hermes.html?utm\\_source=chatgpt.com](https://hermes.informal.systems/documentation/configuration/configure-hermes.html?utm_source=chatgpt.com)

17 Installation | Hermes Agent

[https://hermes-agent.nousresearch.com/docs/getting-started/installation/?utm\\_source=chatgpt.com](https://hermes-agent.nousresearch.com/docs/getting-started/installation/?utm_source=chatgpt.com)

19 33 46 MCP Config Reference | Hermes Agent

<https://hermes-agent.nousresearch.com/docs/reference/mcp-config-reference/>

24 Connect Claude Code to tools via MCP - Claude Docs

[https://docs.claude.com/en/docs/claude-code/mcp?utm\\_source=chatgpt.com](https://docs.claude.com/en/docs/claude-code/mcp?utm_source=chatgpt.com)

26 1. Command line and environment — Python 3.13.7 documentation

[https://docs.python.org/3/using/cmdline.html?highlight=pythondontwritebytecode&utm\\_source=chatgpt.com](https://docs.python.org/3/using/cmdline.html?highlight=pythondontwritebytecode&utm_source=chatgpt.com)

27 sys — System-specific parameters and functions — Python 3.10.20 documentation

[https://docs.python.org/3.10/library/sys.html?utm\\_source=chatgpt.com](https://docs.python.org/3.10/library/sys.html?utm_source=chatgpt.com)

28 chcp | Microsoft Learn

[https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/chcp?utm\\_source=chatgpt.com](https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/chcp?utm_source=chatgpt.com)

30 37 How to Set Up Model Context Protocol (MCP) in Continue | Continue Docs

<https://docs.continue.dev/customize/deep-dives/mcp>

31 Adding & Configuring Servers - Cline

<https://docs.cline.bot/mcp/adding-and-configuring-servers>

34 MCP\_SMOKE\_TEST.md

[https://github.com/pzychozen/TORMENT/blob/main/docs/MCP\\_SMOKE\\_TEST.md](https://github.com/pzychozen/TORMENT/blob/main/docs/MCP_SMOKE_TEST.md)

35 MCP\_README.md

[https://github.com/pzychozen/TORMENT/blob/main/docs/MCP\\_README.md](https://github.com/pzychozen/TORMENT/blob/main/docs/MCP_README.md)

36 **Connect Claude Code to tools via MCP - Claude Code Docs**

[https://code.claude.com/docs/en/mcp?utm\\_source=chatgpt.com](https://code.claude.com/docs/en/mcp?utm_source=chatgpt.com)

38 **The MCP Registry - Model Context Protocol**

<https://modelcontextprotocol.io/registry/about>

40 45 48 **Mem0 - The Memory Layer for your AI Apps**

[https://mem0.ai/?utm\\_source=chatgpt.com](https://mem0.ai/?utm_source=chatgpt.com)

49 **Memory | Zep Documentation**

[https://help.getzep.com/v2/memory?utm\\_source=chatgpt.com](https://help.getzep.com/v2/memory?utm_source=chatgpt.com)

50 **Agent memory & architecture | Letta Docs**

[https://docs.letta.com/guides/agents/architectures/memgpt?utm\\_source=chatgpt.com](https://docs.letta.com/guides/agents/architectures/memgpt?utm_source=chatgpt.com)

51 **Cognee - AI Memory MCP for Developers: Bridging Models to Memory**

[https://www.cognee.ai/blog/cognee-news/introducing-cognee-mcp?utm\\_source=chatgpt.com](https://www.cognee.ai/blog/cognee-news/introducing-cognee-mcp?utm_source=chatgpt.com)

52 **Hermes Agent + Honcho - Honcho**

[https://docs.honcho.dev/v3/guides/integrations/hermes?utm\\_source=chatgpt.com](https://docs.honcho.dev/v3/guides/integrations/hermes?utm_source=chatgpt.com)

53 **Memory overview - Docs by LangChain**

[https://docs.langchain.com/oss/python/concepts/memory?utm\\_source=chatgpt.com](https://docs.langchain.com/oss/python/concepts/memory?utm_source=chatgpt.com)

54 **Memory - LlamaIndex**

[https://docs.llamaindex.ai/en/stable/module\\_guides/deploying/agents/memory/?utm\\_source=chatgpt.com](https://docs.llamaindex.ai/en/stable/module_guides/deploying/agents/memory/?utm_source=chatgpt.com)

55 **Configuration | Hermes Agent**

[https://hermes-agent.nousresearch.com/docs/user-guide/configuration/?utm\\_source=chatgpt.com](https://hermes-agent.nousresearch.com/docs/user-guide/configuration/?utm_source=chatgpt.com)